Building a world-class orbital constellation manager dashboard

The key to a competition-winning orbital dashboard lies in five interlocking systems: a photorealistic Earth with custom GLSL day/night shaders, a particle pipeline that renders 100k+ debris objects in a single draw call, satellite.js-powered coordinate transforms from ECI to screen space, a glassmorphic dark-theme CSS grid layout, and robust edge-case handling for fuel depletion, Kessler cascades, and degenerate conjunction geometries. This report provides production-ready code for every layer of the stack—shaders, React Three Fiber components, CSS design tokens, orbital math, and defensive algorithms.

Photorealistic Earth with day/night terminator and atmosphere glow

The visual centerpiece is a custom `ShaderMaterial` that blends NASA day and night-lights textures based on the dot product between each surface fragment's normal and the sun direction vector. The critical detail is using `smoothstep(-0.15, 0.25, NdotL)` for a wide, realistic twilight zone rather than a hard terminator edge.

Complete Earth fragment shader (GLSL):

```
uniform sampler2D uDayTexture;
uniform sampler2D uNightTexture;
uniform sampler2D uSpecularMap;
uniform sampler2D uBumpMap;
uniform vec3 uSunDirection;
uniform float uBumpStrength;

varying vec2 vUv;
varying vec3 vNormal;
varying vec3 vPosition;

void main() {
  vec3 dayColor   = texture2D(uDayTexture, vUv).rgb;
  vec3 nightColor = texture2D(uNightTexture, vUv).rgb * 1.5;
  float specMask  = texture2D(uSpecularMap, vUv).r;
  float bumpVal   = texture2D(uBumpMap, vUv).r;
```

```
  // Bump perturbation via screen-space derivatives
  float bumpDx = dFdx(bumpVal) * uBumpStrength;
  float bumpDy = dFdy(bumpVal) * uBumpStrength;
  vec3 normal = normalize(vNormal + bumpDx * normalize(dFdx(vPosition))
                          + bumpDy * normalize(dFdy(vPosition)));

  float NdotL = dot(normal, uSunDirection);
  float dayMix = smoothstep(-0.15, 0.25, NdotL);
  float diffuse = max(NdotL, 0.0);
  vec3 color = mix(nightColor, dayColor * (0.3 + 0.7 * diffuse), dayMix);

  // Blinn-Phong specular on oceans only
  vec3 viewDir = normalize(cameraPosition - vPosition);
  vec3 halfVec = normalize(uSunDirection + viewDir);
  float specular = pow(max(dot(normal, halfVec), 0.0), 64.0) * specMask * dayMix;
  color += vec3(1.0, 0.95, 0.8) * specular * 0.7;

  // Twilight tint (warm orange-red at terminator)
  float twilight = pow(clamp(1.0 - abs(NdotL * 4.0), 0.0, 1.0), 3.0);
  color += vec3(1.0, 0.4, 0.2) * twilight * 0.08;

  // Fresnel atmospheric rim
  float fresnel = pow(1.0 - max(dot(viewDir, normal), 0.0), 3.0);
  color += vec3(0.4, 0.6, 1.0) * fresnel * dayMix * 0.12;

  gl_FragColor = vec4(color, 1.0);
}
```

The atmosphere glow renders on a **12% larger sphere** with `THREE.BackSide` and additive blending. The Fresnel effect—`dot(viewDir, -normal)` raised to power **3.5**—creates the characteristic thin blue rim. Sun-facing modulation keeps a **15% base glow** on the dark side while brightening the sunlit limb.

```
// Atmosphere fragment shader (key lines)
float fresnel = pow(clamp(dot(viewDir, -normal), 0.0, 1.0), uFresnelPower);
float sunMix = 0.15 + 0.85 * smoothstep(-0.5, 1.0, dot(-normal, uSunDirection));
gl_FragColor = vec4(uAtmosphereColor, fresnel * uAtmosphereIntensity * sunMix);
```

The cloud layer is a separate sphere at **1.008×** Earth radius with its own shader, rotating independently at a slightly faster rate to simulate atmospheric drift. Renderer configuration uses `ACESFilmicToneMapping` with exposure **1.2** and `SRGBColorSpace` output. Best free texture sources are Solar System Scope (CC BY 4.0) for 2K–8K Earth maps, NASA Blue/Black Marble

for public-domain alternatives, and live-cloud-maps.pages.dev for real-time cloud alpha maps refreshed every 3 hours.

Rendering 100k+ debris objects at 60fps with React Three Fiber

Three architectural tiers handle different object counts, each optimized for a single draw call.

Tier 1 — Points + BufferGeometry for debris clouds (100k–1M objects):

```
export function DebrisField({ count = 100000 }: { count?: number }) {
  const pointsRef = useRef<THREE.Points>(null!);

  const { positions, colors, sizes } = useMemo(() => {
    const positions = new Float32Array(count * 3);
    const colors = new Float32Array(count * 3);
    const sizes = new Float32Array(count);
    for (let i = 0; i < count; i++) {
      const theta = Math.random() * Math.PI * 2;
      const phi = Math.acos(2 * Math.random() - 1);
      const r = 1.05 + Math.random() * 0.6; // LEO to MEO shell
      positions[i * 3]     = r * Math.sin(phi) * Math.cos(theta);
      positions[i * 3 + 1] = r * Math.sin(phi) * Math.sin(theta);
      positions[i * 3 + 2] = r * Math.cos(phi);
      const brightness = 0.3 + Math.random() * 0.7;
      colors[i * 3] = brightness; colors[i * 3+1] = brightness * 0.9; colors[i * 3+2] = brightness * 0.8;
      sizes[i] = 0.5 + Math.random() * 2.0;
    }
    return { positions, colors, sizes };
  }, [count]);

  useFrame((_state, delta) => {
    pointsRef.current.rotation.y += delta * 0.02;
  });

  return (
    <points ref={pointsRef} frustumCulled={false}>
      <bufferGeometry>
        <bufferAttribute attach="attributes-position" count={count} array={positions} itemSize={3} />
        <bufferAttribute attach="attributes-color" count={count} array={colors} itemSize={3} />
        <bufferAttribute attach="attributes-size" count={count} array={sizes} itemSize={1} />
      </bufferGeometry>
```

```
      <pointsMaterial size={1.5} sizeAttenuation vertexColors transparent
        opacity={0.8} depthWrite={false} blending={THREE.AdditiveBlending} />
    </points>
  );
}
```

Tier 2 — InstancedMesh for satellites needing orientation (10k–100k):

```
const dummy = new THREE.Object3D(); // Declared OUTSIDE component — avoids GC

export function SatelliteSwarm({ count = 5000 }: { count?: number }) {
  const meshRef = useRef<THREE.InstancedMesh>(null!);
  const satellites = useMemo(() =>
    Array.from({ length: count }, () => ({
      theta: Math.random() * Math.PI * 2,
      phi: Math.acos(2 * Math.random() - 1),
      radius: 1.06 + Math.random() * 0.12,
      speed: 0.001 + Math.random() * 0.005,
    })), [count]);

  useFrame((_s, delta) => {
    satellites.forEach((sat, i) => {
      sat.theta += sat.speed * delta * 60;
      dummy.position.set(
        sat.radius * Math.sin(sat.phi) * Math.cos(sat.theta),
        sat.radius * Math.sin(sat.phi) * Math.sin(sat.theta),
        sat.radius * Math.cos(sat.phi));
      dummy.lookAt(0, 0, 0);
      dummy.updateMatrix();
      meshRef.current.setMatrixAt(i, dummy.matrix);
    });
    meshRef.current.instanceMatrix.needsUpdate = true;
  });

  return (
    <instancedMesh ref={meshRef} args={[null, null, count]} frustumCulled={false}>
      <octahedronGeometry args={[0.008, 0]} />
      <meshStandardMaterial metalness={0.8} roughness={0.2} />
    </instancedMesh>
  );
}
```

Tier 3 — Custom glow shader for debris particles:

```
// Fragment shader — soft glowing circle with exponential falloff
void main() {
  vec2 center = gl_PointCoord - vec2(0.5);
  float dist = length(center);
  if (dist > 0.5) discard;
  float strength = pow(1.0 - dist * 2.0, 1.5);
  float pulse = 0.9 + 0.1 * sin(uTime * 2.0 + vSize * 10.0);
  gl_FragColor = vec4(uColor * strength * pulse, strength * uOpacity);
}
```

The performance hierarchy is clear: **Points** gives 1 draw call for 1M particles; **InstancedMesh** gives 1 draw call for 100k oriented objects; individual meshes max out around 1,000. The critical `useFrame` rules: never call `setState` (use refs), multiply all motion by `delta`, pre-allocate reusable `Vector3`/`Matrix4`/`Color` objects outside components, and use frame-skipping (`frameCount % 3 === 0`) for expensive non-visual updates like collision checks.

ECI coordinates to Three.js scene and ground track projection

The coordinate transform between satellite.js ECI output (Z-up, km) and Three.js (Y-up, arbitrary units) is the foundation of all orbital visualization.

```
const EARTH_RADIUS_KM = 6371;
const SCALE = 1 / EARTH_RADIUS_KM; // 1 scene unit = 1 Earth radius

function eciToThreeJs(eci) {
  return new THREE.Vector3(
    eci.x * SCALE,
    eci.z * SCALE,    // ECI Z (north) → Three.js Y (up)
    -eci.y * SCALE    // ECI Y → Three.js -Z
  );
}

// Earth rotation for ECI mode
const gmst = satellite.gstime(new Date());
earthMesh.rotation.y = gmst;
```

Ground track with antimeridian handling is the trickiest 2D problem. The key is detecting longitude jumps exceeding 180° and splitting the polyline into separate segments:

```
function drawGroundTrack(ctx, points, W, H) {
  const segments = [[]];
  for (let i = 0; i < points.length; i++) {
    if (i > 0 && Math.abs(points[i].lon - points[i-1].lon) > 180)
      segments.push([]);
    const x = (points[i].lon + 180) * (W / 360);
    const y = (90 - points[i].lat) * (H / 180);
    segments.at(-1).push({ x, y });
  }
  segments.forEach(seg => {
    if (seg.length < 2) return;
    ctx.beginPath();
    ctx.moveTo(seg[0].x, seg[0].y);
    seg.slice(1).forEach(p => ctx.lineTo(p.x, p.y));
    ctx.stroke();
  });
}
```

For **orbit trail rendering**, `Line2` (fat lines from three/addons) provides pixel-width control with vertex-color gradients for fading trails. `MeshLine` adds width tapering. For thousands of orbits, basic `THREE.Line` with `LineBasicMaterial` is lightest. The satellite footprint circle uses the angular radius formula `θ = acos(R_E / (R_E + alt))` and traces a great circle at that angular distance from the sub-satellite point.

Conjunction visualization uses a 2D bullseye plot in the encounter plane (perpendicular to relative velocity). The primary object sits at the origin, combined hard-body radius (HBR) is drawn as a red circle, covariance error ellipses at 1σ/2σ/3σ surround it, and the secondary object's projected miss distance is plotted as a marker. Color coding follows NASA CARA thresholds: **Pc > 10^{-4} red**, **> 10^{-5} orange**, **> 10^{-7} yellow**, below that green.

Interactive satellite selection with raycasting and labels

Points raycasting requires a threshold because points have no geometric volume. Set it on the Canvas raycaster:

```
<Canvas raycaster={{ params: { Points: { threshold: 0.05 } } }}>
  <ClickableDebris />
</Canvas>

function ClickableDebris() {
  const handleClick = useCallback((event: ThreeEvent<MouseEvent>) => {
```

```
    event.stopPropagation();
    const idx = event.index; // Which point vertex was hit
    if (idx === undefined) return;
    // Read position from geometry attribute
    const pos = pointsRef.current.geometry.attributes.position;
    selectSatellite({ x: pos.getX(idx), y: pos.getY(idx), z: pos.getZ(idx) });
  }, []);

  return <points onClick={handleClick}>...</points>;
}
```

For **InstancedMesh**, R3F provides `event.instanceId` automatically. Always call `event.stopPropagation()` to receive only the frontmost hit rather than all instances along the ray.

Drei's `<Html>` component creates DOM-based tooltips that track 3D positions with occlusion support:

```
<Html position={[0, 1.5, 0]} center distanceFactor={20} occlude
  style={{ background: 'rgba(0,10,30,0.85)', border: '1px solid rgba(100,180,255,0.4)',
    borderRadius: '8px', padding: '8px 12px', fontFamily: 'monospace',
    backdropFilter: 'blur(8px)', boxShadow: '0 0 15px rgba(50,120,255,0.3)' }}>
  <div style={{ fontWeight: 'bold', color: '#64b5f6' }}>{name}</div>
  <div>Alt: {altitude.toFixed(1)} km</div>
</Html>
```

Use `<Html>` for the **1 currently-selected** satellite's rich tooltip. For bulk labels, use drei's `<Billboard>` + `<Text>` (renders as SDF text in WebGL, handles 500+ labels). Architecture rule of thumb: Points for 10k+ distant debris, InstancedMesh for oriented satellites, Html for the selected satellite's detail panel, Billboard+Text for nearby labels.

Mission control dark-theme CSS design system

The design system is built entirely on CSS custom properties for rapid theming. The background color `#060b18` provides the essential near-black-with-blue-tint that reads as "space." Every interactive element uses **layered glow effects** via `box-shadow` and `text-shadow`.

```
:root {
  --bg-space: #060b18;
  --bg-panel: rgba(8, 16, 32, 0.75);
```

```
  --bg-elevated: rgba(12, 22, 45, 0.85);
  --cyan: #00d4ff;
  --amber: #ffa500;
  --red: #ff3b3b;
  --green: #00e676;
  --text-primary: rgba(224, 240, 255, 0.95);
  --text-muted: rgba(160, 180, 200, 0.5);
  --glow-cyan: 0 0 8px rgba(0,212,255,0.35), 0 0 20px rgba(0,212,255,0.15);
  --glow-red:  0 0 8px rgba(255,59,59,0.35), 0 0 20px rgba(255,59,59,0.15);
  --font-mono: 'JetBrains Mono', 'Fira Code', monospace;
}
```

CSS Grid layout places the 3D canvas as the primary surface with glass-morphic panels overlaid:

```
.dashboard {
  display: grid;
  grid-template-columns: 280px 1fr 320px;
  grid-template-rows: 1fr 200px;
  grid-template-areas:
    "sidebar  canvas  telemetry"
    "sidebar  timeline telemetry";
  height: 100vh;
  background: var(--bg-space);
}

.panel {
  background: var(--bg-panel);
  backdrop-filter: blur(12px) saturate(1.4);
  border: 1px solid rgba(100, 180, 255, 0.08);
  border-radius: 8px;
  z-index: 10; /* Panels float above the canvas (z-index: 0) */
}
```

Animated status indicators use pulsing box-shadows for operational awareness:

```
.status-dot { width: 8px; height: 8px; border-radius: 50%; position: relative; }
.status-dot--nominal { background: var(--green); }
.status-dot--nominal::after {
  content: ''; position: absolute; inset: -4px; border-radius: 50%;
  border: 2px solid var(--green); opacity: 0;
  animation: status-pulse 2s ease-out infinite;
}
```

```
@keyframes status-pulse {
  0% { transform: scale(0.8); opacity: 0.6; }
  100% { transform: scale(2.2); opacity: 0; }
}
.status-dot--critical { background: var(--red); }
.status-dot--critical::after { border-color: var(--red); animation-duration: 1s; }
```

Telemetry values use `font-variant-numeric: tabular-nums` with monospace fonts so numbers don't shift as digits change. Value updates trigger a brief cyan flash animation (`@keyframes telemetry-flash`) that fades back to the base color in 400ms. Conjunction event cards use a left-edge severity stripe via `::before` pseudo-element — red for critical (Pc > 10^{-4}), amber for warning, green for nominal. Toast notifications slide in from the right with a cubic-bezier ease and include an auto-dismiss progress bar animated via `@keyframes toast-progress`.

Edge cases that separate good submissions from great ones

Fuel depletion triggers a state machine transition

When `fuel_remaining < stationkeeping_cost`, the satellite transitions from CONTROLLED → UNCONTROLLED → DECAYED. The drag coefficient jumps from **~2.2 (stabilized) to ~2.5–3.0 (tumbling)** as attitude control is lost, cross-sectional area increases, and orbital decay accelerates exponentially. Below **300 km**, atmospheric density increases so rapidly that reentry occurs within days. The critical cascade effect: a fuel-depleted satellite becomes a conjunction threat to the rest of the constellation and must immediately trigger re-screening.

Near-zero miss distance requires numerical safeguards

The standard 2D Pc calculation divides by `√det(C)` where C is the combined covariance. As miss distance approaches zero, three failure modes emerge: degenerate (singular) covariance causes division by zero, non-positive-definite covariance matrices (common in real operational data) produce imaginary eigenvalues, and the exponential integral overflows. NASA CARA handles this with a multi-step algorithm: **2D-Pc (94.7% of cases)** → **2D-Nc (5.0%)** → **3D-Nc (0.3%)** → **Monte Carlo (<0.01%)**. Always check `det(C) > ε`, remediate NPD matrices by clipping negative eigenvalues, and clamp output Pc to [0, 1].

Large time steps destroy SGP4 accuracy

SGP4 in-track error grows at **~1–3 km/day** for LEO objects. After 7 days, errors reach **25–50 km** — conjunction screening becomes meaningless. The implementation must subdivide any span >5 minutes into steps of ≤300 seconds, track TLE epoch age (warn at 7 days, reject at 14), and **inflate covariance proportionally** to TLE age using a quadratic growth model: `scale = 1 + (days/3)²`. The Kepler equation solver needs a bisection fallback for high-eccentricity orbits (e > 0.9) where Newton-Raphson fails to converge.

Uncorrelated tracks demand a correlation-first pipeline

New telemetry from unknown objects follows: validate → attempt catalog correlation (Mahalanobis distance against existing objects) → if uncorrelated, attempt initial orbit determination by associating with other pending UCTs via Lambert solver → insert provisionally with inflated covariance. A sudden burst of UCTs in similar orbital regimes signals a fragmentation event and must trigger a breakup alert.

Kessler cascade requires computational triage

A catastrophic collision (specific energy > **40 J/g**) generates fragments following the NASA Standard Breakup Model: `N(Lc) = 0.1 × M^0.75 × Lc^(-1.71)`. A 1,000 kg satellite hit at 10 km/s produces **thousands** of trackable fragments. The O(N²) conjunction screening becomes infeasible without spatial indexing — use a **kd-tree or octree** for neighbor queries. Cap simulated fragments (e.g., 10,000 with statistical weights) and prioritize conjunctions by miss distance and consequence. The key detail judges look for: acknowledging that the NASA SBM does **not conserve mass, momentum, or energy** (Finkleman & Oltrogge, 2008).

Conclusion: the integration architecture

The winning architecture connects these systems through an **event-driven pipeline**: satellite.js propagates TLEs in a Web Worker → ECI positions flow through the coordinate transform to both the 3D scene (Points/InstancedMesh) and the 2D ground-track canvas → conjunction screening runs every N frames via the frame-skipping pattern → state changes (fuel depletion, new UCTs, fragmentation events) propagate through a Zustand store accessed via `getState()` inside `useFrame` (never `useState`). The renderer pipeline stacks: star field → Earth with day/night shader → atmosphere glow → cloud layer → debris Points → satellite InstancedMesh → orbit trail Line2 → selected satellite Html tooltip. Every layer has been designed to minimize draw calls and keep the frame budget under **16.6ms** even with 100k+ objects on screen.

Project AETHER: Complete Technical Research Report

The most critical finding across all six research areas: your hackathon stack of React Three Fiber + FastAPI can comfortably handle 10,000+ debris objects at 10 Hz with the right architecture patterns. InstancedMesh renders 5,000+ particles in a single draw call, scipy's KDTree screens conjunctions in under 5 ms, and NumPy vectorized propagation processes 10,000 orbits in under 0.5 ms per tick. Below is every code pattern, formula, threshold value, and CSS hex code you need.

3D globe rendering with Three.js and WebGL

Earth with day/night blend shader

The standard approach uses a `SphereGeometry` (64×64 segments) with a custom `ShaderMaterial` that blends NASA day and night textures based on sun direction. The fragment shader uses a sigmoid function for smooth twilight transition:

```
// earth.frag
uniform sampler2D u_dayTexture;
uniform sampler2D u_nightTexture;
uniform sampler2D u_specTexture;
uniform vec3 u_sunDirection;
varying vec2 vUv;
varying vec3 vNormal;
varying vec3 vPosition;

void main() {
   vec3 dayColor = texture2D(u_dayTexture, vUv).rgb;
   vec3 nightColor = texture2D(u_nightTexture, vUv).rgb;
   float cosAngle = dot(normalize(vNormal), normalize(u_sunDirection));
   float mixAmount = 1.0 / (1.0 + exp(-20.0 * cosAngle));  // sigmoid twilight
   vec3 color = mix(nightColor, dayColor, mixAmount);

   // Specular highlights for oceans
   float specMap = texture2D(u_specTexture, vUv).r;
   vec3 viewDir = normalize(cameraPosition - vPosition);
   vec3 reflectDir = reflect(-normalize(u_sunDirection), normalize(vNormal));
   float spec = pow(max(dot(viewDir, reflectDir), 0.0), 16.0);
```

```
  color += mixAmount * spec * specMap * 0.4;
  gl_FragColor = vec4(color, 1.0);
}
```

Texture sources include NASA Blue Marble (day), NASA Earth at Night (emissive), and Solar System Scope (CC BY 4.0). Compress with **KTX2/Basis** format for GPU efficiency; keep textures at 4096×2048 max for web. Combine specular and cloud data into a single texture (R = specular, G = clouds) to reduce GPU memory.

Atmosphere glow via backface Fresnel

The most performant atmosphere technique renders a slightly larger sphere (scale **1.15×**) with `side: THREE.BackSide` and additive blending. The Fresnel effect creates intensity that increases at the edges:

```
<mesh scale={[1.15, 1.15, 1.15]}>
  <sphereGeometry args={[1, 64, 64]} />
  <shaderMaterial
    vertexShader={atmosphereVert}
    fragmentShader={`
      uniform vec3 uColor;
      uniform float uIntensity;
      varying vec3 vNormal;
      varying vec3 vPosition;
      void main() {
        vec3 viewDir = normalize(-vPosition);
        float fresnel = dot(viewDir, vNormal);
        float atmosphere = pow(fresnel, 1.5) * uIntensity;
        gl_FragColor = vec4(uColor, atmosphere);
      }
    `}
    uniforms={{
      uColor: { value: new THREE.Color('#4444ff') },
      uIntensity: { value: 1.2 },
    }}
    side={THREE.BackSide}
    transparent
    blending={THREE.AdditiveBlending}
    depthWrite={false}
  />
</mesh>
```

The **Stemkoski glow parameter tuning** provides three presets: glow (FrontSide, c=0.05, p=4.5), halo/atmosphere (BackSide, c=0.6, p=6), and shell (FrontSide, c=1, p=2). Set `depthWrite={false}` to avoid occluding stars behind the atmosphere.

Cloud layer

Add a separate sphere at scale **1.005×** with a semi-transparent cloud texture:

```
<mesh scale={[1.005, 1.005, 1.005]}>
  <sphereGeometry args={[1, 64, 64]} />
  <meshPhongMaterial map={cloudTexture} transparent opacity={0.4} depthWrite={false} />
</mesh>
```

Rotate the cloud sphere slightly faster than Earth for realism.

Instanced rendering for 5,000+ debris particles

InstancedMesh is the clear winner

For 5,000+ identical debris objects, `InstancedMesh` renders everything in a **single draw call** versus 5,000+ separate draw calls. Declare the dummy `Object3D` outside the component to avoid garbage collection:

```
const dummy = new THREE.Object3D()

function DebrisField({ count = 5000, particleData }) {
  const meshRef = useRef()

  useLayoutEffect(() => {
    for (let i = 0; i < count; i++) {
      dummy.position.set(particleData[i].x, particleData[i].y, particleData[i].z)
      dummy.scale.setScalar(particleData[i].size || 0.02)
      dummy.updateMatrix()
      meshRef.current.setMatrixAt(i, dummy.matrix)
    }
    meshRef.current.instanceMatrix.needsUpdate = true
  }, [count])

  useFrame((state, delta) => {
    for (let i = 0; i < count; i++) {
      meshRef.current.getMatrixAt(i, dummy.matrix)
```

```
      dummy.matrix.decompose(dummy.position, dummy.quaternion, dummy.scale)
      dummy.position.applyAxisAngle(particleData[i].orbitAxis, particleData[i].speed * delta)
      dummy.updateMatrix()
      meshRef.current.setMatrixAt(i, dummy.matrix)
    }
    meshRef.current.instanceMatrix.needsUpdate = true
  })

  return (
    <instancedMesh ref={meshRef} args={[undefined, undefined, count]}>
      <icosahedronGeometry args={[1, 0]} />  {/* Low-poly for debris */}
      <meshBasicMaterial color="#aaaaaa" />
    </instancedMesh>
  )
}
```

For **per-instance colors** (color-coding debris by risk level), use `InstancedBufferAttribute` on the geometry and set `vertexColors` on the material. For **>50,000 particles** where you only need dots, switch to `THREE.Points` with a custom `ShaderMaterial` — zero per-vertex geometry overhead.

Critical performance rules: use `useLayoutEffect` (not `useEffect`) for initial matrix setup; use low-poly geometry like `icosahedronGeometry args={[1, 0]}`; avoid drei's `<Instances>` + `<Instance>` components for counts above 5,000 since they create React elements per instance; set `meshRef.current.count = actualVisibleCount` to render fewer instances without recreating.

Orbit trails with BufferGeometry

For dynamic satellite trails, preallocate a `Float32Array` and use `setDrawRange` to grow the trail:

```
function OrbitTrail({ maxPoints = 200 }) {
  const lineRef = useRef()
  const positions = useMemo(() => new Float32Array(maxPoints * 3), [])
  const currentIndex = useRef(0)

  useFrame(() => {
    const pos = satellite.getPosition()
    const idx = currentIndex.current % maxPoints
    positions[idx * 3] = pos.x
    positions[idx * 3 + 1] = pos.y
    positions[idx * 3 + 2] = pos.z
```

```
    currentIndex.current++
    lineRef.current.geometry.attributes.position.needsUpdate = true
    lineRef.current.geometry.setDrawRange(0, Math.min(currentIndex.current, maxPoints))
  })

  return (
    <line_ ref={lineRef}>
      <bufferGeometry>
        <bufferAttribute attach="attributes-position" count={maxPoints} array={positions}
itemSize={3} />
      </bufferGeometry>
      <lineBasicMaterial vertexColors transparent opacity={0.8} />
    </line_>
  )
}
```

For static Keplerian orbits, precompute all 360 points using the vis-viva equation and render as a drei `<Line>`. For flickering/z-fighting at large orbital scales, enable `logarithmicDepthBuffer: true` on the Canvas.

Post-processing bloom for space glow

```
import { EffectComposer, Bloom, Vignette, Noise } from '@react-three/postprocessing'

<EffectComposer>
  <Bloom luminanceThreshold={1} intensity={2} mipmapBlur radius={0.8} />
  <Vignette offset={0.3} darkness={0.9} />
  <Noise opacity={0.03} blendFunction={BlendFunction.OVERLAY} />
</EffectComposer>
```

Selective bloom works by setting `luminanceThreshold={1}` — only objects with `emissiveIntensity > 1` and `toneMapped={false}` will glow. This is simpler and more performant than `SelectiveBloom` with refs. Use `mipmapBlur` for better quality at lower cost than large kernel sizes.

Orbital mechanics formulas and edge cases

Collision geometry and closing speeds

The relative velocity between objects depends on encounter geometry. At **800 km LEO** (orbital velocity ≈ 7.5 km/s):

- **Head-on (counter-orbital, θ=180°)**: **~15 km/s** (2 × v_orbit) — the hardest to avoid
- **Perpendicular crossing (θ=90°)**: **~10.6 km/s** (√2 × v_orbit)
- **Co-planar trailing (θ≈0°)**: **~0 km/s** — the 2D Pc assumption breaks down here
- **Mean collision velocity in LEO**: **~10 km/s** (weighted by actual inclination distributions)

The 2009 Iridium 33/Cosmos 2251 collision occurred at **11.7 km/s**. At 15 km/s, objects traverse 1 km in just **0.067 seconds** — TCA resolution must be sub-second.

TCA detection algorithms

The time of closest approach satisfies **r_rel · v_rel = 0** (relative position perpendicular to relative velocity). Three key algorithms exist:

- **ANCAS** (Alfano-Negron): Fits cubic polynomials to relative velocity, finds roots analytically. Fast but can miss global minima.
- **CATCH** (Chebyshev Proxy Polynomials): Uses Chebyshev approximation with Boyd's eigenvalue-based root finding on 16×16 companion matrices. Comparable speed to ANCAS with better accuracy.
- **Smart Sieve** (Rodriguez/Klinkrad, ESA 2002): Cascade filter architecture — apogee-perigee filter → orbit path filter → time filter → rectangular overlap test → fine detection.

For your hackathon, implement the **cascade filter** approach: first check altitude overlap (trivial), then plane intersection, then time filter, then precise distance computation.

Tsiolkovsky rocket equation for fuel tracking

$\Delta v = Isp \times g_0 \times \ln(m_0/mf)$
Propellant mass = $mf \times (e^{(\Delta v/(Isp \times g_0))} - 1)$

Where **g_0 = 9.80665 m/s²**. Key Isp values for your simulation:

- **Monopropellant hydrazine**: Isp = **220 s**, thrust 0.5–22 N (most common for LEO maneuvering)
- **Hall-effect thruster**: Isp = **1,500–2,000 s**, thrust 40–300 mN (electric propulsion)
- **Cold gas (N_2)**: Isp = **65 s** (CubeSat thrusters)

A typical collision avoidance maneuver requires **2–50 mm/s** delta-v. For a 500 kg satellite with Isp=220s hydrazine, a 5 cm/s maneuver uses approximately **11.6 g of propellant** per burn. Total avoidance cost is roughly **2× the avoidance delta-v** (avoidance + recovery burn).

J2 perturbation propagation

The **J_2 coefficient = 1.08264 × 10^{-3}** dominates all other gravitational perturbations (~100× larger than J_3). The Cartesian acceleration equations:

$$a_J2_x = -(3/2) \times J_2 \times \mu \times R_E^2 \times (x/r^5) \times (1 - 5z^2/r^2)$$
$$a_J2_y = -(3/2) \times J_2 \times \mu \times R_E^2 \times (y/r^5) \times (1 - 5z^2/r^2)$$
$$a_J2_z = -(3/2) \times J_2 \times \mu \times R_E^2 \times (z/r^5) \times (3 - 5z^2/r^2)$$

Where **μ = 398600.4418 km^3/s^2** and **R_E = 6378.137 km**. Over 24 hours, RAAN precession reaches **up to 7–8° per day** in LEO. SGP4 propagation error grows at **~1–3 km/day**, dominated by along-track uncertainty. For a 24-hour conjunction assessment horizon, J2-only propagation gives meaningful improvement over two-body but won't match SGP4 accuracy for drag-affected low orbits.

CDM risk thresholds and Pc computation

Real-world operational thresholds

These are the actual thresholds used operationally by space agencies:

- **ISS (human spaceflight)**: Pc ≥ **1×10^{-5}** triggers notification; Pc ≥ **1×10^{-4}** triggers maneuver. Screening box: ±2 km radial × 25 km × 25 km along/cross-track.
- **NASA robotic missions**: Pc ≥ **1×10^{-4}** (1 in 10,000) triggers maneuver planning via CARA.
- **ESA**: Maneuver threshold at Pc ≥ **1×10^{-4}** operationally; CNES uses a "plausibilistic" approach with 1×10^{-4} to 5×10^{-4}.
- **Post-maneuver target**: Reduce Pc to below **1×10^{-7}**.
- **Lifetime collision limit** (NASA-STD-8719.14B): Pc < **0.001** for objects >10 cm over entire mission.

NASA CARA classifies events with Pc > **4.4×10^{-4}** as "Red" requiring active operator engagement. CDMs are generated approximately 3 times per day, with screening extending up to 7 days before TCA.

Probability of collision from miss distance and covariance

The standard 2D Pc integral evaluates a bivariate Gaussian over a circular hard-body region in the encounter plane (perpendicular to relative velocity):

$$Pc = \iint_HBR\ (1/(2\pi|C|^{1/2})) \times \exp(-\tfrac{1}{2} \times r^T \times C^{-1} \times r)\ dr$$

Where **$C = C_1 + C_2$** (combined 2×2 covariance projected onto the encounter plane) and **HBR** = sum of both objects' effective radii. The maximum possible Pc for given covariance is **$Pc_max = HBR^2 / (2 \times \sigma_x \times \sigma_y)$**. Chan's method provides an efficient approximate closed-form series solution; Foster's method uses direct numerical integration. For co-orbital (low relative velocity) encounters, the 2D linearization fails — use Monte Carlo instead.

Evasion maneuver timing

The operational timeline follows this sequence:

- **7 days out**: Initial screening, CDM generation begins
- **3 days out**: Alert if Pc ≥ planning threshold, intensive CDM monitoring
- **24 hours**: Go/No-Go decision (NASA standard)
- **6–24 hours before TCA**: Burn execution
- **8 hours**: ESA operational minimum decision time
- **0.5–1 orbit after TCA**: Recovery burn

The key relationship for along-track burns: **separation grows linearly with lead time** ($\delta_T \approx 3 \times \Delta t \times \Delta v$). Earlier maneuvers require less delta-v for the same miss distance. ESA's Envisat example: 4.0 cm/s braking burn applied half a revolution (~50 minutes) before TCA, recovery burn of 3.9 cm/s half a revolution after. Total fuel cost: **~7.9 cm/s** for the complete avoidance+recovery cycle.

FastAPI backend architecture for 10 Hz telemetry

Core architecture: background simulation thread + WebSocket push

The optimal pattern uses a daemon thread running the simulation loop, shared state protected by `threading.Lock`, and WebSocket with **msgpack binary frames** for client updates:

```
class SimulationEngine:
    def __init__(self, n_debris=10000, n_sats=100, dt=0.1):
        self._lock = threading.Lock()
        self.debris_states = np.zeros((n_debris, 6), dtype=np.float32)  # [x,y,z,vx,vy,vz]
        self.sat_states = np.zeros((n_sats, 6), dtype=np.float64)

    def step(self):
        with self._lock:
            # Vectorized propagation — 0.1-0.5ms for 10K objects
            positions = self.debris_states[:, :3]
            velocities = self.debris_states[:, 3:]
            r_mag = np.linalg.norm(positions, axis=1, keepdims=True)
```

```
acc = -398600.4418 * positions / np.maximum(r_mag ** 3, 1e-10)
velocities += 0.5 * acc * self.dt
positions += velocities * self.dt
# Recompute acceleration, complete Velocity Verlet...

# KDTree conjunction screening — 1-3ms for 10K points
tree = KDTree(self.debris_states[:, :3], leafsize=16)
self.conjunctions = tree.query_ball_point(
    self.sat_states[:, :3], r=10.0, workers=-1
)
```

Performance budget at 10 Hz (100 ms/tick): vectorized propagation takes ~0.5 ms, KDTree build ~3 ms, conjunction query ~1 ms, msgpack serialization ~1 ms. Total: **~5 ms per tick** — well within budget.

KDTree method selection

Use `query_ball_point` for your use case of 100 satellites against 10,000 debris (build tree from the larger set, query with the smaller). The `workers=-1` parameter enables all CPU cores. `query_pairs` is better for all-pairs screening within a single set. `query_ball_tree` is best when both sets are large.

Avoiding async event loop blocking

NumPy releases the GIL during C-level computation, so `threading.Lock` works correctly. Three patterns ranked by simplicity:

- **`def` (sync) endpoints**: FastAPI auto-runs these in a threadpool — simplest for NumPy-heavy endpoints
- **`asyncio.to_thread()`**: Explicit offloading for mixed async/sync code
- **`ProcessPoolExecutor`**: True parallelism for heavy CPU work, but requires serializable arguments (copy arrays)

Use `ORJSONResponse` as the default response class for **20–50% faster** JSON serialization. For WebSocket binary frames, msgpack encodes numerical data **1,000× faster** than JSON for NumPy arrays.

Space mission-control visual design system

Complete color palette

```
:root {
  /* Backgrounds */
  --bg-void:       #080d16;   /* Deepest space black with blue undertone */
  --bg-space:      #0a0e17;   /* Primary app background */
  --bg-panel:      #0d1117;   /* Panel/card backgrounds */
  --bg-surface:    #111827;   /* Elevated surfaces */
  --bg-elevated:   #1a1f2e;   /* Modals, popovers */

  /* Neon accents */
  --neon-cyan:     #00e5ff;   /* Primary accent — dominant in space UIs */
  --neon-green:    #39ff14;   /* Success/nominal glow */
  --neon-magenta:  #ff00ff;   /* Secondary accent */

  /* Status colors */
  --status-green:  #00c853;   /* Nominal */
  --warning-amber: #ffab00;   /* Warning */
  --danger-red:    #ff1744;   /* Critical alert */

  /* Text */
  --text-primary:  #e6f3ff;   /* Cool white with blue tint */
  --text-secondary: #8892b0;  /* Muted labels */
  --text-data:     #00e5ff;   /* Data readout values */

  /* Borders */
  --panel-border:  rgba(0, 229, 255, 0.12);
  --border-accent: rgba(0, 229, 255, 0.4);
}
```

The Arwes sci-fi framework uses **#26dafd** as its primary cyan, slightly warmer than `#00e5ff`. Both work well — pick one and stay consistent.

Typography stack

```
:root {
  --font-display: 'Orbitron', sans-serif;          /* Headers — geometric tech font */
  --font-ui: 'Rajdhani', 'Exo 2', 'Inter', system-ui;  /* Body/UI labels */
  --font-mono: 'JetBrains Mono', 'Fira Code', monospace; /* ALL numerical data */
}
```

The single most distinctive trait of space UIs is **all-caps labels with wide letter-spacing** (`letter-spacing: 0.1em; text-transform: uppercase`). Use monospace for all telemetry readouts to ensure vertical digit alignment. NASA Open MCT and SpaceX Crew

Dragon (which runs on Chromium/HTML/JS) both use compact font sizes — body text at **14px**, not 16px.

Panel styling with HUD corner accents

```
.panel {
  background: rgba(13, 17, 23, 0.85);
  backdrop-filter: blur(12px);
  border: 1px solid rgba(0, 229, 255, 0.12);
  border-radius: 4px;
  padding: 16px 20px;
  box-shadow: 0 0 20px rgba(0, 229, 255, 0.05), 0 4px 16px rgba(0, 0, 0, 0.4);
  position: relative;
}
.panel::before {
  content: '';
  position: absolute;
  top: 0; left: 0;
  width: 24px; height: 24px;
  border-top: 2px solid var(--neon-cyan);
  border-left: 2px solid var(--neon-cyan);
  opacity: 0.6;
}
.panel::after {
  content: '';
  position: absolute;
  bottom: 0; right: 0;
  width: 24px; height: 24px;
  border-bottom: 2px solid var(--neon-cyan);
  border-right: 2px solid var(--neon-cyan);
  opacity: 0.6;
}
```

Layout: overlay panels on 3D scene

The standard pattern for space visualization dashboards overlays UI panels on the full-viewport 3D scene:

```
.scene-overlay {
  position: fixed;
  inset: 0;
  display: grid;
  grid-template-columns: 320px 1fr 320px;
  grid-template-rows: 56px 1fr auto;
```

```
  pointer-events: none;   /* Click through to 3D scene */
  z-index: 10;
}
.scene-overlay > * {
  pointer-events: auto;   /* Re-enable on actual panels */
}
```

Top bar holds mission clock, status indicators, and mode controls. Left panel holds navigation/satellite tree. Right panel holds telemetry readouts and conjunction alerts. Bottom bar holds timeline and simulation speed controls.

Tailwind configuration for the full design system

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        void: '#080d16', space: '#0a0e17', surface: '#111827',
        panel: '#0d1117', elevated: '#1a1f2e',
        neon: { cyan: '#00e5ff', green: '#39ff14', magenta: '#ff00ff' },
        status: { green: '#00c853', amber: '#ffab00', red: '#ff1744' },
        muted: { DEFAULT: '#8892b0', dark: '#5a6a8a' },
      },
      fontFamily: {
        display: ['Orbitron', 'sans-serif'],
        ui: ['Rajdhani', 'Exo 2', 'Inter', 'system-ui'],
        mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
      },
      letterSpacing: { 'space-wide': '0.05em', 'space-wider': '0.1em', 'space-widest': '0.2em' },
      boxShadow: {
        'glow-cyan': '0 0 8px rgba(0,229,255,0.15), 0 0 24px rgba(0,229,255,0.08)',
        'glow-red': '0 0 8px rgba(255,23,68,0.2), 0 0 24px rgba(255,23,68,0.1)',
        'neon': '0 0 6px rgba(0,229,255,0.25), inset 0 0 6px rgba(0,229,255,0.15)',
      },
    },
  },
}
```

Recommended npm packages

Package	Purpose
`three` ^0.167+	Core 3D engine
`@react-three/fiber` ^8.x	React renderer for Three.js
`@react-three/drei` ^9.x	Helpers: OrbitControls, Line, Trail, Stars, useTexture
`@react-three/postprocessing` ^3.0	Bloom, ChromaticAberration, Vignette
`postprocessing` ^6.37+	Underlying post-processing library
`r3f-perf`	Performance monitor overlay
`leva`	Live parameter tuning GUI (invaluable for hackathons)
`zustand`	State management (pmndrs ecosystem)
`vite-plugin-glsl`	Import .glsl/.vert/.frag files in Vite

Python backend: `fastapi[standard]`, `uvicorn[standard]`, `orjson`, `msgpack`, `scipy`, `numpy`, `uvloop`, `httptools`.

Conclusion

Three architectural decisions will determine Project AETHER's success at NSH 2026. First, **use InstancedMesh with a single draw call** for all debris — never create individual React components for thousands of objects. Second, **run orbital propagation in a background thread** with vectorized NumPy, not in async endpoints — the entire 10K-object simulation fits comfortably in a 5 ms tick. Third, **the 2D Pc integral with a cascade screening filter** (apogee-perigee → plane intersection → KDTree proximity → precise TCA) gives you operationally realistic conjunction assessment without requiring a full astrodynamics library. The design system built around `#080d16` backgrounds, `#00e5ff` cyan accents, and Orbitron/JetBrains Mono typography will immediately read as "mission control" to judges. Enable `logarithmicDepthBuffer` on the R3F Canvas from day one — orbital scale differences will cause z-fighting otherwise, and this is the kind of bug that wastes hours during a hackathon.

Orbital debris avoidance system: complete hackathon technical reference

This guide delivers concrete, implementation-ready answers across five critical subsystems for building a satellite conjunction detection and visualization platform. The core takeaway: **use a cascading filter pipeline (apogee-perigee → MOID → coarse propagation → fine TCA) to reduce N² screening to manageable pairs**, render debris as `THREE.Points` with `BufferGeometry` for 60fps at 10k+ objects, and compute evasion burns with the Clohessy-Wiltshire state transition matrix — not the naive Δv = miss/time formula.

---

1. Conjunction screening: the cascading filter pipeline

Professional systems like NASA CARA and CelesTrak's SOCRATES never run detailed propagation on every satellite-debris pair. With **25,000 tracked objects producing ~312 million pairs**, they use a multi-gate filter cascade that eliminates >99.9% of pairs before expensive computation begins.

Screening volumes and thresholds

NASA CARA screens LEO conjunctions using an ellipsoidal volume of **0.5 km (radial) × 17 km (in-track) × 20 km (cross-track)** in the RIC frame, looking 7 days ahead for LEO and 10 days for GEO/HEO. SOCRATES uses a simpler **5 km spherical threshold** at TCA with SGP4 propagation. For a hackathon with a 24-hour prediction horizon, **5–10 km spherical is practical for TLE-based screening**.

Alert thresholds follow a tiered model: **$Pc < 10^{-7}$** requires no action, **$Pc \geq 10^{-4}$** triggers mitigation planning, and **$Pc \geq 4.4\times10^{-4}$** demands maneuver execution for NASA robotic missions. The ISS uses a more conservative **$Pc \geq 10^{-5}$** threshold for human spaceflight.

The five filter gates

```
FULL CATALOG (N objects, ~N²/2 pairs)
   │
   ▼
[GATE 1: Apogee-Perigee overlap]     ← Eliminates ~90-95% of pairs
   │
   ▼
[GATE 2: RAAN proximity filter]      ← Eliminates ~50-70% of remainder
   │
   ▼
```

```
[GATE 3: MOID orbit-path filter]     ← Eliminates ~80-90% of remainder
    │
    ▼
[GATE 4: Coarse propagation, 60s steps] ← Eliminates ~95-99%
    │
    ▼
[GATE 5: Fine TCA refinement, 1s steps] ← ~500-5000 final conjunctions
```

Gate 1 (apogee-perigee) is the single biggest performance win and should be implemented first. Use *mean* orbital elements (not osculating) to avoid false negatives from J2 short-period variations:

```
def ap_filter(obj1, obj2, pad_km=10.0):
    """Returns True if altitude bands overlap (conjunction possible)."""
    r_peri_1 = obj1.a * (1 - obj1.e)
    r_apo_1  = obj1.a * (1 + obj1.e)
    r_peri_2 = obj2.a * (1 - obj2.e)
    r_apo_2  = obj2.a * (1 + obj2.e)
    return (min(r_apo_1, r_apo_2) + pad_km) >= (max(r_peri_1, r_peri_2) - pad_km)
```

For spatial indexing at scale, the most practical hackathon approach is sorting all objects by perigee altitude (O(N log N)), then only comparing objects within overlapping altitude bands. For position-space screening at discrete time steps, `scipy.spatial.KDTree` provides fast nearest-neighbor queries:

```
from scipy.spatial import KDTree

positions = propagate_all_sgp4(catalog, epoch)  # shape (N, 3) in km
tree = KDTree(positions)
candidate_pairs = tree.query_pairs(r=50.0)  # 50 km coarse threshold
# Refine only these pairs with fine-step propagation
```

---

2. Three.js orbital visualization: from points to atmosphere

Rendering 5000+ debris as a GPU point cloud

Individual meshes create per-object draw calls and kill performance. **`THREE.Points` with `BufferGeometry` renders 10,000+ objects in a single draw call** at 60fps:

```
function DebrisCloud({ debrisEciPositions }) {
  const SCALE = 1 / 6371; // normalize so Earth radius = 1

  const positions = useMemo(() => {
    const arr = new Float32Array(debrisEciPositions.length * 3);
    debrisEciPositions.forEach((p, i) => {
      arr[i * 3]     = p.x * SCALE;
      arr[i * 3 + 1] = p.z * SCALE;  // ECI Z → Three.js Y (north = up)
      arr[i * 3 + 2] = p.y * SCALE;  // ECI Y → Three.js Z
    });
    return arr;
  }, [debrisEciPositions]);

  return (
    <points>
      <bufferGeometry>
        <bufferAttribute attach="attributes-position"
          count={positions.length / 3} array={positions} itemSize={3} />
      </bufferGeometry>
      <pointsMaterial size={0.02} color="#ff4444"
        sizeAttenuation transparent opacity={0.8} depthWrite={false} />
    </points>
  );
}
```

For **per-point color coding** (e.g., red = high risk, green = safe), add a second `bufferAttribute` for `color` with `vertexColors={true}` on the material. Use `InstancedMesh` only when you need raycasting/click events on individual debris pieces.

ECI-to-Three.js coordinate mapping

ECI has X toward vernal equinox, Y completing the equatorial plane, Z toward North Pole. **The cleanest mapping for "north = up" scenes** swaps Y and Z:

```
// ECI → Three.js (North Pole = Y-up)
threeX = eci.x / 6371;  // ECI X → Three.js X
threeY = eci.z / 6371;  // ECI Z (north) → Three.js Y (up)
threeZ = eci.y / 6371;  // ECI Y → Three.js Z
```

The scale factor **1/6371** (dividing by Earth's radius in km) normalizes the scene so Earth has radius 1.0. LEO satellites at 400 km appear at radius ~1.063.

Fading orbital trail with per-vertex alpha

Pre-compute one full orbit (~180 points at 30-second intervals for LEO's ~90-min period) as static geometry. Use a custom `ShaderMaterial` for the fade:

```
const material = new THREE.ShaderMaterial({
  transparent: true, depthWrite: false,
  uniforms: { color: { value: new THREE.Color(0x00ffaa) } },
  vertexShader: `
    attribute float alpha;
    varying float vAlpha;
    void main() {
      vAlpha = alpha;
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }`,
  fragmentShader: `
    uniform vec3 color;
    varying float vAlpha;
    void main() { gl_FragColor = vec4(color, vAlpha); }`
});
```

Set the `alpha` attribute from 0.0 at the trail tail to 1.0 at the satellite's current position. Only update the satellite marker position each frame — the trail geometry stays static and is cheap.

Atmospheric glow via back-face Fresnel shader

The standard technique used by virtually every production Earth demo: a **slightly larger sphere rendered on `BackSide` with additive blending** and a Fresnel-based intensity calculation:

```
function Atmosphere() {
  const vertShader = `
    varying vec3 vNormal;
    void main() {
      vNormal = normalize(normalMatrix * normal);
      gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }`;
  const fragShader = `
    varying vec3 vNormal;
    void main() {
      float intensity = pow(0.7 - dot(vNormal, vec3(0,0,1)), 2.0);
      gl_FragColor = vec4(0.3, 0.6, 1.0, 1.0) * intensity;
    }`;
```

```
  return (
    <mesh scale={[1.15, 1.15, 1.15]}>
      <sphereGeometry args={[1, 64, 64]} />
      <shaderMaterial vertexShader={vertShader} fragmentShader={fragShader}
        blending={THREE.AdditiveBlending} side={THREE.BackSide}
        transparent depthWrite={false} />
    </mesh>
  );
}
```

The key insight: `dot(vNormal, vec3(0,0,1))` approaches zero at the limb (grazing angles), so `1 - dot(...)` raised to a power creates the rim glow. The 1.15 scale factor controls glow thickness. For extra polish, add `UnrealBloomPass` post-processing with **strength 0.5, radius 0.4, threshold 0.85**.

Pulsing conjunction warning line

Use a custom shader with a `time` uniform for the pulse animation. For visible thickness, layer a thin bright line with a wider translucent `MeshLine`:

```
function ConjunctionWarning({ posA, posB }) {
  const matRef = useRef();
  useFrame(({ clock }) => {
    const pulse = 0.5 + 0.5 * Math.sin(clock.getElapsedTime() * 5);
    matRef.current.opacity = 0.3 + 0.7 * pulse;
    matRef.current.color.setRGB(1, pulse * 0.3, 0);
  });
  const geom = useMemo(() =>
    new THREE.BufferGeometry().setFromPoints([posA, posB]), [posA, posB]);
  return (
    <line geometry={geom}>
      <lineBasicMaterial ref={matRef} color="red" transparent
        depthWrite={false} blending={THREE.AdditiveBlending} />
    </line>
  );
}
```

Earth rotation synchronized to real GMST

In an ECI-frame scene, satellites are positioned in inertial coordinates and **the Earth texture rotates underneath them**. Apply the Greenwich Mean Sidereal Time as Y-rotation:

```
import * as satellite from 'satellite.js';

function Earth({ simulationTime }) {
  const ref = useRef();
  useFrame(() => {
    const gmst = satellite.gstime(simulationTime || new Date());
    ref.current.rotation.y = gmst;  // aligns texture longitude to ECI frame
  });
  return (
    <mesh ref={ref}>
      <sphereGeometry args={[1, 64, 64]} />
      <meshStandardMaterial map={earthTexture} />
    </mesh>
  );
}
```

You may need a small constant offset (e.g., `-Math.PI/2`) depending on your texture's UV origin to align Greenwich to the correct longitude.

---

3. What makes production Earth visualizations look great

The `impacts-apps/impacts-apps.github.io` repository is not publicly accessible (private or removed). However, analysis of comparable production visualizations (GitHub Globe, satellitetracker3d.com, SCAD3D, three.js Earth examples) reveals a consistent set of techniques that create the polished look:

Five to seven texture layers form the foundation: day map (8K+), night/city lights map, normal/bump map for terrain relief, specular map (ocean reflectivity mask), and a semi-transparent cloud layer. NASA's Visible Earth collection provides free high-resolution source textures. The **day/night shader** uses `smoothstep(-0.2, 0.3, dot(normal, sunDirection))` for a realistic soft terminator blend rather than a hard shadow cutoff.

Renderer configuration matters enormously: `ACESFilmicToneMapping` produces cinematic contrast, `sRGBEncoding` ensures correct color display, and `logarithmicDepthBuffer: true` is essential when rendering objects at vastly different distances (Earth surface vs. GEO at 35,786 km). Multiple lights — a primary directional "sun" light, soft ambient fill, and optional hemisphere light — create depth and dimension. The GitHub Globe team confirmed their approach: "we point four lights at a sphere."

Post-processing bloom (`UnrealBloomPass`) makes the atmosphere glow bleed naturally and gives city lights on the night side a realistic luminosity. Adaptive quality degradation (reducing pixel ratio and geometry count when FPS drops below ~55) ensures cross-device performance.

---

4. FastAPI edge cases that will break naive implementations

Input validation that judges will test

Auto-testers will send **malformed TLEs** (wrong checksums, eccentricity ≥ 1, negative mean motion), **decayed satellite TLEs** (SGP4 error code 6 when position goes underground), **stale epochs** (TLE from years ago propagated forward), and **injection attacks** in satellite name fields. Build a robust Pydantic model:

```
class TLEInput(BaseModel):
    line1: str
    line2: str
    name: str | None = None

    @field_validator('line1', 'line2')
    @classmethod
    def validate_tle_format(cls, v):
        if len(v) != 69: raise ValueError(f"TLE line must be 69 chars, got {len(v)}")
        computed_checksum = sum(int(c) if c.isdigit() else (1 if c == '-' else 0)
                                for c in v[:68]) % 10
        if computed_checksum != int(v[68]):
            raise ValueError(f"Checksum mismatch: computed {computed_checksum}")
        return v

    @model_validator(mode='after')
    def validate_physics(self):
        ecc = float('0.' + self.line2[26:33])
        mean_motion = float(self.line2[52:63])
        if ecc >= 1.0: raise ValueError("Eccentricity >= 1 (hyperbolic)")
        if mean_motion <= 0: raise ValueError("Non-positive mean motion")
        return self
```

Always **handle SGP4 error codes** (1–6) gracefully. Error code 6 (satellite decayed) is the most common — return an informative 422 response, don't let it crash.

Numerical precision for sub-second TCA

When TCA is very close to the current time, **Julian date precision becomes the bottleneck**. A naive `jd = 2460000.5` representation uses 7 digits before the decimal, leaving only ~8–9 for the fractional day. One second = 1.157×10^{-5} days, so you get only ~4 digits of sub-second precision. The fix: **use SGP4's split Julian date** `(jd, fr)` where `jd` holds the integer+0.5 part and `fr` holds the fractional day separately, preserving full double precision.

For TCA refinement, use a three-phase approach: coarse search at 60-second steps, cubic spline interpolation on the distance function, then `scipy.optimize.minimize_scalar` on the spline to find the exact minimum. This achieves **sub-0.1-second TCA accuracy**.

RK4 propagator instability

RK4 is **not symplectic** — it introduces secular energy drift that causes orbits to spiral inward or outward over time. For a hackathon-scale 24-hour propagation this is manageable, but monitor it:

- **Step sizes**: dt < P/100 (LEO: 10–30s, GEO: 60–120s)
- **Eccentric orbits are dangerous**: fixed-step RK4 must use the perigee-dictated step everywhere
- **J2 implementation pitfall**: the z-component uses `(3 - 5·sin²φ)` while x,y use `(1 - 5·sin²φ)` — sign errors here are extremely common

```
def j2_acceleration(r_vec, mu=398600.4418, J2=1.08263e-3, R_E=6378.137):
    x, y, z = r_vec
    r = np.linalg.norm(r_vec)
    factor = -1.5 * J2 * mu * R_E**2 / r**5
    z_r_sq = (z / r) ** 2
    return np.array([
        factor * x * (1 - 5 * z_r_sq),
        factor * y * (1 - 5 * z_r_sq),
        factor * z * (3 - 5 * z_r_sq)   # Note: 3, not 1
    ])
```

Pragmatic recommendation: use SGP4 for TLE-based propagation (it's the *only* correct propagator for mean elements in TLEs). Reserve custom RK4/RK45 for state-vector-based scenarios. For adaptive stepping, `scipy.integrate.solve_ivp` with `method='DOP853'` handles orbital dynamics well out of the box.

---

5. RTN evasion burns: the correct Clohessy-Wiltshire formulation

Why the naive formula fails

The formula `ΔV = target_miss / time_to_TCA` treats space as free of gravity. In reality, the **Clohessy-Wiltshire equations** show that radial and in-track motion are coupled through Coriolis terms, and in-track burns produce secular drift that grows with time — not a linear displacement. A radial impulse generates in-track drift; an in-track impulse produces radial oscillations *plus* growing along-track separation. The naive formula ignores all of this and can underestimate required ΔV by an order of magnitude for multi-orbit lead times.

The CW state transition matrix

For relative motion near a circular reference orbit with mean motion $\mathbf{n = \sqrt{\mu/a^3}}$, the position at TCA given an impulsive ΔV at time t_0 is:

$$r(TCA) = \Phi_rr \cdot r_0 + \Phi_rv \cdot (v_0 + \Delta V)$$

where Φ_rv (the velocity-to-position mapping, the key matrix for maneuver planning) is:

$$\Phi_rv = \begin{bmatrix} \sin(n\tau)/n & 2(1-\cos(n\tau))/n & 0 \\ -2(1-\cos(n\tau))/n & (4\sin(n\tau)-3n\tau)/n & 0 \\ 0 & 0 & \sin(n\tau)/n \end{bmatrix}$$

with τ = time to TCA in seconds, c = cos(nτ), s = sin(nτ).

The minimum-fuel solution

The required ΔV to shift the miss position by δr at TCA is $\mathbf{\Delta V = \Phi_rv^{-1} \cdot \delta r}$. To minimize |ΔV| while achieving standoff distance d_target:

1. Compute the cost metric $\mathbf{A = (\Phi_rv^{-1})^T \cdot \Phi_rv^{-1}}$
2. Eigendecompose A — the **eigenvector with minimum eigenvalue** is the most fuel-efficient direction to shift miss position
3. Offset along that direction to reach the d_target sphere

```
import numpy as np
from numpy.linalg import inv, norm, eigh

def compute_cw_stm(n, dt):
    """CW state transition submatrices. n=mean motion (rad/s), dt=seconds to TCA."""
```

```
    tau = n * dt
    c, s = np.cos(tau), np.sin(tau)
    Phi_rr = np.array([
        [4 - 3*c,       0, 0],
        [6*(s - tau),   1, 0],
        [0,             0, c]])
    Phi_rv = np.array([
        [s/n,           2*(1 - c)/n,       0    ],
        [-2*(1 - c)/n,  (4*s - 3*tau)/n,    0    ],
        [0,             0,                  s/n  ]])
    return Phi_rr, Phi_rv

def minimum_dv_evasion(n, dt, r_rel_rtn, v_rel_rtn, d_target=2000.0):
    """
    Compute minimum-ΔV evasion burn in RTN frame.
    n: mean motion (rad/s), dt: seconds to TCA
    r_rel_rtn, v_rel_rtn: relative state in RTN (meters, m/s)
    d_target: desired miss distance (meters)
    Returns: (dv_rtn [m/s], dv_magnitude, post_burn_miss)
    """
    Phi_rr, Phi_rv = compute_cw_stm(n, dt)

    # Predicted miss without burn
    r_tca_noburn = Phi_rr @ r_rel_rtn + Phi_rv @ v_rel_rtn
    if norm(r_tca_noburn) >= d_target:
        return np.zeros(3), 0.0, norm(r_tca_noburn)

    # Find cheapest direction via eigenvalue decomposition
    Phi_rv_inv = inv(Phi_rv)
    A = Phi_rv_inv.T @ Phi_rv_inv
    eigenvalues, eigenvectors = eigh(A)
    best_dir = eigenvectors[:, np.argmin(eigenvalues)]  # cheapest direction

    # Solve |r_noburn + α·best_dir| = d_target
    b = np.dot(r_tca_noburn, best_dir)
    c_coef = norm(r_tca_noburn)**2 - d_target**2
    disc = b**2 - c_coef
    alpha1 = -b + np.sqrt(max(0, disc))
    alpha2 = -b - np.sqrt(max(0, disc))
    alpha = alpha1 if abs(alpha1) < abs(alpha2) else alpha2

    delta_r = alpha * best_dir
    dv = Phi_rv_inv @ delta_r
    return dv, norm(dv), norm(r_tca_noburn + delta_r)
```

Why in-track burns dominate

The Φ_rv (2,2) element — `(4sin(nτ) - 3nτ)/n` — contains the **secular term -3τ** that grows linearly with lead time. For lead times exceeding one orbit, this dominates all other elements. Practically: **an in-track burn 5 hours before TCA at 550 km altitude produces ~25 km along-track offset per 0.4 m/s of ΔV**. Radial burns produce only bounded oscillations (max displacement ~2ΔV/n ≈ a few km) with no secular growth. Cross-track burns are independent simple harmonic motion with maximum effect at quarter-orbit before TCA.

The ECI-to-RTN rotation matrix to convert your computed ΔV back to inertial coordinates:

```
def eci_to_rtn(r_eci, v_eci):
    """Returns 3x3 rotation matrix. Rows = R_hat, T_hat, N_hat."""
    R_hat = r_eci / norm(r_eci)
    h = np.cross(r_eci, v_eci)
    N_hat = h / norm(h)
    T_hat = np.cross(N_hat, R_hat)
    return np.array([R_hat, T_hat, N_hat])

# To convert RTN ΔV to ECI: dv_eci = R.T @ dv_rtn
```

Conclusion

The five subsystems described here form a complete, implementable architecture. The conjunction screening pipeline's apogee-perigee gate alone eliminates 90–95% of the computational burden, making real-time screening of thousands of objects feasible even in Python. On the visualization side, the combination of `THREE.Points` for debris, Fresnel back-face atmosphere, and GMST-synchronized Earth rotation creates a professional result with minimal code. The CW state transition matrix approach to evasion burns is the industry standard — it properly accounts for the coupling between radial and in-track dynamics that the naive formula ignores, and the eigenvalue decomposition finds the genuinely fuel-optimal direction rather than assuming a purely transverse burn. For robustness, validate TLEs aggressively, use split Julian dates for sub-second precision, and monitor RK4 energy drift as a propagation sanity check.